

Tailwind CSS for This Project

A from-scratch introduction to Tailwind CSS, written for someone who already knows CSS well but has never used Tailwind — grounded entirely in how it's actually used in this codebase.

You already know everything except the naming. Tailwind doesn't add new styling powers CSS doesn't already have — `display: flex` is still `display: flex`. What changes is that instead of inventing a class name and writing a rule for it in a stylesheet, you reach for a class name Tailwind has already defined for you, and write it directly on the element. This document walks through that idea, then the three specific ways *this* project bends the default Tailwind style — because it ported an existing pixel-perfect design rather than starting a new one from Tailwind's default scale.

Contents

1. The core idea: rules become class names — the mental shift
2. Reading a class name — anatomy of a utility
3. How the CSS actually gets built — content scanning, the 3 directives
4. Cheat-sheet: what this project actually uses — grouped by CSS property
5. This project's twist #1 — arbitrary values — why 1,700+ square brackets
6. This project's twist #2 — custom color tokens — how theming works without `dark:`
7. This project's twist #3 — when classes aren't enough — the inline-style escape hatch
8. A real component, fully annotated — class by class
9. Where to look in the code — the three config files

1. The core idea: rules become class names

In hand-written CSS, you name a concept, then define it once:

PLAIN CSS — THE WAY YOU KNOW

```
.toolbar {  
  display: flex;  
  align-items: center;  
  gap: 8px;  
  padding: 12px;  
}
```

...then in HTML: `<div class="toolbar">`

TAILWIND — ONE CLASS PER PROPERTY

```
<div class="flex items-center gap-2  
p-3">
```

No `.toolbar` rule to invent or maintain.
Each word is a complete, ready-made utility: one word, one CSS declaration.

Both approaches produce the exact same computed styles. The difference is entirely about *where you look to know what an element looks like*: with plain CSS, you check the markup for a class name, then jump to a stylesheet (maybe a different file) to see what it means. With Tailwind, the markup *is* the answer — every property affecting that element is listed right there, in the `class` attribute, in plain sight.

This trade-off has a real cost worth naming honestly: the `className` strings get long, and they can look noisy at first. What you get back is that you never have to invent a name for something (is it `.card`, `.card-panel`, `.panel-box`?), and you never end up with a stylesheet containing ten near-identical rules that drifted apart over time — every element's styling is self-contained.

2. Reading a class name — anatomy of a utility

Nearly every Tailwind class you'll see in this codebase follows one shape. Take a real one from `renderer/src/features/shell/ScreenHeader.tsx`:

hover	:	border-acc
STATE	SEPARATOR	PROPERTY + VALUE

`hover:border-acc` means: *"when this element is hovered, set its border color to the value named `acc`."* Drop the `hover:` prefix and `border-acc` alone means the same thing unconditionally. Other prefixes work the same way — `disabled:`, `focus:`, `last:` (used in `Settings.tsx` to drop the border on the last row of a list) all exist in this project. Two prefixes that *don't* appear anywhere in this codebase are worth knowing about anyway, precisely because their absence is a deliberate choice:

PREFIX FAMILY	USED HERE?	WHY
sm: md: lg: xl:	No (0 occurrences)	These pick different styles at different <i>browser widths</i> — essential for a public website viewed on phones and desktops. FluencyOS is a desktop app with one window and a defined minimum size (<code>minWidth: 960</code> in <code>electron/main/index.ts</code>), so there's no "mobile layout" to design for.
dark:	No (0 occurrences)	Tailwind's built-in dark-mode prefix swaps a class outright (<code>bg-white dark:bg-black</code>). This project instead lets the same class (e.g. <code>bg-panel</code>) resolve to a different <i>value</i> depending on which theme's CSS variables are active — see Section 6.

3. How the CSS actually gets built

Tailwind doesn't ship a giant stylesheet containing every possible utility (that would be megabytes). Instead, three things work together in this project:

1 · THE DIRECTIVES

`renderer/src/styles/globals.css`

starts with three lines:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Each is later replaced by generated CSS: `base` is a browser reset (like `normalize.css`), `components` is empty here (an escape hatch for custom reusable classes, unused in this project), `utilities` is the actual pile of `.flex`, `.p-3`-style classes.

2 · CONTENT SCANNING

`tailwind.config.ts` lists

which files to scan as plain text for class names:

```
content:
[ './index.html',
  './src/**/*.{ts,tsx}' ]
```

Tailwind never runs your code — it literally text-searches these files for anything that looks like a class name it recognizes, and generates CSS *only* for classes it actually finds. This is why the final CSS file is small despite Tailwind "knowing" thousands of possible utilities.

3 · POSTCSS

`postcss.config.js`

plugs Tailwind into the build as a CSS post-processor. Vite (the dev server / bundler from the main codebase document) runs PostCSS automatically on every `.css` file, so this step needs no manual attention while developing.

The gotcha this implies: because Tailwind finds classes by scanning source text, a class name that only exists at runtime — built by joining strings — can be invisible to it. Section 7 shows exactly where this bites, and how this project avoids it.

4. Cheat-sheet: what this project actually uses

Tailwind ships hundreds of utilities; this project uses a fairly small, consistent subset.

Everything below was found by searching the actual `.tsx` files, so this table is a real map of what you'll encounter reading the code — not the full Tailwind manual.

GROUP	CLASSES YOU'LL SEE	PLAIN CSS EQUIVALENT
Layout	flex, grid, hidden, relative, absolute, overflow-y-auto	<code>display:flex/grid/none</code> , <code>position:relative/absolute</code> , <code>overflow-y:auto</code>
Flex/grid detail	flex-1, flex-none, flex-col, items-center, justify-between, gap-2, grid-cols-4	<code>flex:1</code> , <code>flex:none</code> , <code>flex-direction:column</code> , <code>align-items:center</code> , <code>justify-content:space-between</code> , <code>gap:8px</code> , <code>grid-template-columns:repeat(4,1fr)</code>
Sizing	w-full, h-full, min-w-0, min-h-0, max-w-[660px]	<code>width:100%</code> , <code>height:100%</code> , <code>min-width:0</code> , <code>min-height:0</code> , <code>max-width:660px</code>
Spacing	p-4, px-3, py-2, m-0, mt-[14px], gap-[6px]	<code>padding</code> , <code>padding-left/right</code> , <code>padding-top/bottom</code> , <code>margin</code> , <code>margin-top:14px</code> , <code>gap:6px</code>
Typography	font-sans, font-mono, font-semibold, text-[12.5px], tracking-[-0.02em], leading-[1.6], uppercase, truncate	<code>font-family</code> , <code>font-weight:600</code> , <code>font-size:12.5px</code> , <code>letter-spacing</code> , <code>line-height</code> , <code>text-transform:uppercase</code> , <code>text-overflow:ellipsis;white-space:nowrap;overflow:hidden</code>
Color / background	text-tx, text-tx2, bg-panel, bg-acc, border-line2	<code>color</code> / <code>background</code> / <code>border-color</code> , using this project's own custom color names — see Section 6
Borders & radius	border, border-t, border-dashed, rounded-full, rounded-field, rounded-panel	<code>border-width:1px</code> , <code>border-top-width:1px</code> , <code>border-style:dashed</code> , <code>border-radius:9999px</code> , and two project-specific radii (Section 6)
Effects	shadow-panel, opacity-60, brightness-110 (via <code>hover:</code>)	<code>box-shadow</code> (custom, Section 6), <code>opacity:0.6</code> , <code>filter:brightness(1.1)</code>
Interaction	hover:border-acc, hover:text-acc, disabled:opacity-40	<code>:hover { border-color: ... }</code> , <code>:disabled { opacity: 0.4 }</code> — Tailwind expands the prefix into the matching CSS pseudo-class automatically

5. This project's twist #1 — arbitrary values

A stock Tailwind class like `p-3` maps to a value from a predefined scale (roughly multiples of 4px). That's fine when you're designing fresh and happy to round to that scale. This project instead **ported an existing pixel-perfect mockup** (the static HTML file in `claude-ui-mockup-files/`), so it needs exact values Tailwind's default scale doesn't offer — `13.5px` font sizes, `0.02em` letter-spacing, and so on. Tailwind supports this with square-bracket syntax: **any utility can take a one-off value in brackets instead of a scale keyword**.

DEFAULT SCALE

`p-3` → padding: 12px
`text-sm` → font-size: 14px
`w-64` → width: 16rem

ARBITRARY VALUE — USED THROUGHOUT THIS PROJECT

`p-[14px]` → padding: 14px
`text-[13.5px]` → font-size: 13.5px
`w-[270px]` → width: 270px

This is not a rare escape hatch here — a search across `renderer/src` turns up **over 1,700** square-bracket arbitrary values. That number is a direct fingerprint of "this UI was ported from an exact existing design," and it's worth saying so plainly if asked: a Tailwind project designed from scratch usually leans much more on the default scale, because there's no pre-existing pixel value to match.

```
// real example — renderer/src/features/conversation/Report.tsx
<div className="grid grid-cols-[1.2fr_0.8fr_0.7fr_1fr_1.4fr] gap-[10px] border-b border-line2
py-[10px]">
```

Even `grid-template-columns` — a value with no sensible fixed "scale" at all — takes an arbitrary value the same way: `grid-cols-[1.2fr_0.8fr_...]`. Underscores inside the brackets stand in for spaces (a real space would end the class name), and Tailwind converts them back when generating the CSS.

6. This project's twist #2 — custom color tokens

Stock Tailwind ships color names like `bg-red-500` or `text-slate-700`. This project uses none of those — instead, `tailwind.config.ts` defines its own color names, each pointing at a CSS custom property (variable) rather than a fixed hex code:

```
// renderer/tailwind.config.ts (trimmed)
colors: {
  acc: 'var(--acc)',          // the accent/brand color
  accSoft: 'var(--accSoft)', accLine: 'var(--accLine)',
  bg: 'var(--bg)', bg2: 'var(--bg2)',
  panel: 'var(--panel)', panel2: 'var(--panel2)',
  line: 'var(--line)', line2: 'var(--line2)',
  tx: 'var(--tx)', tx2: 'var(--tx2)', tx3: 'var(--tx3)', // text, at 3 opacity steps
},
borderRadius: { panel: '10px', field: '7px' }, // → rounded-panel, rounded-field
boxShadow: { panel: 'var(--shadow)' }, // → shadow-panel
```

The actual values those variables hold live in `renderer/src/styles/globals.css`, and — this is the important part — **there are two different sets of values, one per theme:**

```
body[data-theme='dark'] { --bg: #131514; --panel: #1e2120; --tx: #e9ebe9; /* ... */ }
body[data-theme='light'] { --bg: #f2f2f0; --panel: #ffffff; --tx: #171a19; /* ... */ }
```

So when JSX writes `className="bg-panel text-tx"`, that compiles to plain CSS referencing the variables (`background: var(--panel); color: var(--tx)`) — Tailwind's part is done at build time and never changes again. What actually changes at runtime is only one attribute, toggled by `shellStore.toggleTheme()` (in the file you have open, `renderer/src/store/shellStore.ts`):

```
document.body.setAttribute('data-theme', next); // 'dark' or 'light'
```

The browser's own CSS cascade does the rest: every element using `bg-panel` anywhere in the app instantly resolves to a different color, because the variable it points to changed — not because any component re-rendered with different class names. This is *why* this project never needed Tailwind's `dark:` prefix from Section 2: the class names never differ between themes, only the values they resolve to.

7. This project's twist #3 — when classes aren't enough

The rule from Section 3, restated as a warning: Tailwind generates CSS by scanning your source files as plain text. A complete class name like `"bg-acc"` is visible to that scan. A class name assembled at runtime, like ``bg-${colorVar}``, is **not** — the scanner never sees the string `"bg-acc"` anywhere in the file, so it never generates that CSS rule, and the style silently fails to apply.

This project hits that exact wall constantly — many screens color an element based on data that only exists at runtime (a score, a due/overdue state, whichever tab is active). The fix used throughout the codebase is: **keep Tailwind for everything static, and drop to a plain inline `style` object for the handful of values that are genuinely dynamic.** Real example, from `renderer/src/features/settings/Settings.tsx`:

```

<button
  className="rounded-field px-[10px] py-2 text-left font-sans text-[12.5px] font-medium
  hover:bg-line2"
  style={{
    background: on ? 'var(--accSoft)' : 'transparent', // depends on component state (`on`)
    color: on ? 'var(--acc)' : 'var(--tx2)',           // → can't be a static class name
  }}
>

```

Everything that is *always true* about this button (its padding, font, corner rounding, hover background) is a plain Tailwind class. The two properties that depend on `on` — a boolean computed while the app is running — go into a regular React `style` prop, which is just a plain JavaScript object mapped straight onto the DOM element's inline `style` attribute, exactly as `element.style.background = '...'` would in plain JS/DOM code you've written before.

You'll see this same split constantly: progress bar widths (`style={{ width: `${pct}%` }}`), per-row colors pulled from a data array, and computed layout values (the Forest screen's isometric tile positions) are all inline styles for exactly this reason — not inconsistency, but working around a real, documented limitation of how Tailwind decides what CSS to generate.

8. A real component, fully annotated

One more pass over the same button from Section 7, this time translating every single class into the exact plain CSS property it produces:

CLASS	PLAIN CSS IT BECOMES
rounded-field	<code>border-radius: 7px;</code> — a name this project defined in <code>tailwind.config.ts</code> , not a stock Tailwind value
px-[10px]	<code>padding-left: 10px; padding-right: 10px;</code>
py-2	<code>padding-top: 8px; padding-bottom: 8px;</code> — Tailwind's default scale (step "2" = 8px)
text-left	<code>text-align: left;</code>
font-sans	<code>font-family: var(--sans);</code> — this project's font stack, from <code>tailwind.config.ts</code> 's <code>fontFamily</code> extension
text-[12.5px]	<code>font-size: 12.5px;</code>
font-medium	<code>font-weight: 500;</code>
hover:bg-line2	<code>&:hover { background: var(--line2); }</code>

Eight classes, eight properties, zero surprises — every word in the class list is exactly one declaration, which is the entire premise of "utility-first CSS" made concrete.

9. Where to look in the code

FILE	WHAT IT'S RESPONSIBLE FOR
renderer/tailwind.config.ts	Which files to scan for class names (<code>content</code>), and every custom name this project adds on top of stock Tailwind — colors, font families, border radii, shadow.
renderer/postcss.config.js	Wires the Tailwind plugin into the CSS build pipeline; you'll rarely need to open this.
renderer/src/styles/globals.css	The three <code>@tailwind</code> directives, plus every CSS variable's actual value for the dark and light themes.

Companion reading: *FluencyOS — Frontend, Electron & Onboarding Explained* covers React, Electron, Vite and TypeScript the same way this document covers Tailwind. All class-usage counts and examples above were taken directly from the current `renderer/src` source, not from Tailwind's general documentation.